SOFTWARE REQUIREMENTS SPECIFICATION

for

RaftKV

A Dynamic Raft-Based Distributed Key-Value Store of N Nodes

Version 1.0

Prepared by:

1. **Anubhav yadav (1024240104)**
2. **Kunwar Aran Srivastava (1024240003)**
3. **Yuvraj Kumar (1024240143)**
4. **Pratik Batish (1024240101)**

Submitted to: Ms. Anushka

October 10, 2026

Contents

1 Introduction 3
1.1 Purpose 3
1.2 Intended Audience and Reading Suggestions 3
1.3 Project Scope 4
2 Overall Description 6
2.1 Product Perspective 6
2.2 User Classes and Characteristics 6
2.3 Product Functions 7
2.4 Operating Environment 10
2.5 Design 11
3 System Features 17
3.1 Description and Priority 17
3.2 Functional Requirements 17
3.2.1 Cluster Size, Quorum and Node Roles 18
3.2.2 Leader Election 18
3.2.3 Log Replication 19
3.2.4 Key-Value State Machine 19
3.2.5 Client API 19
3.2.6 Persistence and Recovery 20
3.2.7 Snapshotting and Compaction 20
3.2.8 Dynamic Membership 21
3.2.9 Fault Injection and Testing Support 22
3.2.10 CLI, Deployment and Observability 22
4 Other Nonfunctional Requirements 23
4.1 Performance Requirements 23
4.2 Security Requirements 23
4.3 Software Quality Attributes 24
4.4 Business Rules 24
5 Other Requirements 25
5.1 Maintenance and Evolution 25
5.2 Out of Scope 25
5.3 Assumptions and Items to Confirm 25
5.4 Glossary 26

1 Introduction

1.1 Purpose

A key-value store that runs on a single machine loses availability, and possibly data, when that machine fails. It is very difficult to keep such data safe by hand, because any machine can crash, any disk can be lost and any node can be added or removed at any time. So, “RaftKV” is the solution. “RaftKV” is a distributed key-value store written in Go that replicates its data across a cluster of **N nodes** using the Raft consensus algorithm. As long as a majority of the N voting nodes are working, the data stays consistent and the store stays available. RaftKV is built from scratch with a real Raft implementation (leader election, log replication, persistence), not a simulation of one.

Two ideas are central to “RaftKV”. First, the cluster is made of N nodes and N is a configuration value, not a constant in the code (3 to 5 nodes in the MVP). Second, the cluster is **dynamic**: it can be reconfigured while it is running. A new node joins as a non-voting learner, catches up with the cluster, and is then promoted to a voting member without downtime, so N grows to N + 1 while clients keep reading and writing. In the same way a node can be removed (a stretch goal), so N can shrink to N − 1. The majority that is needed for an election or a commit is always counted over the voters of the current configuration, that is, $\lfloor N/2 \rfloor + 1$ of the N voters.

This SRS specifies the requirements of the complete, idealised RaftKV system: the minimum viable product (MVP) and the stretch goals. Phases 1 and 2 of the implementation plan have been completed; this document is the reference for what the whole system must do.

1.2 Intended Audience and Reading Suggestions

This SRS is for the developers who own the Raft core, storage, transport, API and membership modules, for testers, for the project supervisor, and for reviewers. Further the discussion will provide all the internal, external, functional and also non-functional informations about “RaftKV”.

- Everyone: read sections 1 and 2 for the overview and the architecture.
- Developers and testers: section 3 holds the numbered functional requirements (FR-n); section 4 holds the nonfunctional requirements (NFR-n).
- Reviewers: section 5.3 lists the assumptions and the items still to be confirmed.

Priority keywords. MUST = required for the MVP. SHOULD = important, attempted after the MVP is stable (stretch). MAY = optional stretch. OUT_OF_SCOPE = will not be built.

1.3 Project Scope

“RaftKV” creates a space for Application Clients and Cluster Operators to store, read and delete key-value data in a replicated cluster of N nodes, and to change the membership of that cluster, that is the value of N, while it is running.

A client sends a PUT, GET or DELETE request over HTTP/JSON to any node. If that node is not the leader, a write is redirected to the leader. The leader appends the command to its log, replicates it to the other N − 1 nodes and, when a majority of the voters hold it, commits and applies it to the key-value state machine. Only then the client gets the answer. If the leader or any minority of nodes crashes, the remaining majority elects a new leader and no committed data is lost.

An operator can use the `raftctl` command to see the cluster status and to add a new node. The new node is first a learner: it receives the log (or a snapshot) but does not vote and is not counted for the majority. When it has caught up, the leader promotes it to a voter through a committed configuration entry, and the cluster has N + 1 voters. Removal of a node follows the same path in the other direction.

In scope (MVP):

- A cluster of N nodes (3 to 5 in the MVP) with leader election and log replication; N is set by configuration and can change at runtime.
- PUT, GET and DELETE over HTTP/JSON, with redirect of writes that reach a non-leader.
- Durable write-ahead log (WAL) and crash recovery.
- Basic snapshotting and log compaction.
- Dynamic node join through a learner stage and promotion to voter (N to N + 1).
- Fault injection covering leader crash, follower crash, network partition and node restart.
- Docker Compose deployment of an N-node cluster, CI, and core documentation.

Stretch: safe node removal (N to N − 1), linearizable reads (read-index or leader-lease), chunked snapshot transfer, deeper metrics, benchmarking, and optional features (watches, TTL/leases, compare-and-set, simple transactions).

Out of scope: TLS/mTLS between nodes, RBAC, encryption at rest, audit logging, Kubernetes manifests, cloud deployment automation, multi-host orchestration. See section 5.2.

Implementation status. Phases 1 and 2 are implemented. The phase plan was not part of the material used to write this version, so the requirements are tagged by priority and not by phase. See section 5.3.

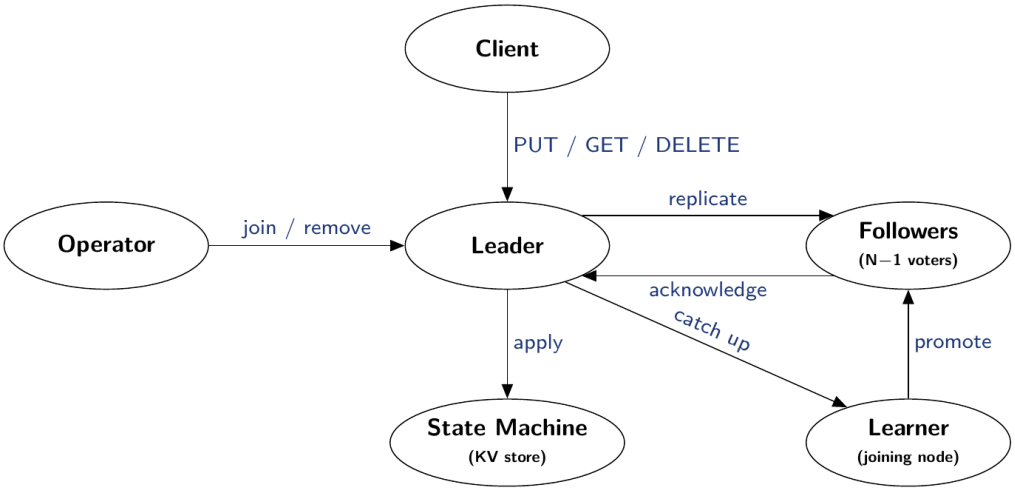


Figure 1.1: Entire work-flow

Figure 1.1 (Entire work-flow) is the overview of the project. Connection of all the entities are dependable to each others. This gives the simple idea about the functional activities of the project.

Client cycle, In the Figure 1.1 “Client” sends requests to the “Leader”; “Leader” replicates them to “Followers” and waits for their acknowledgement; “Leader” applies the committed command to the “State Machine”.

Membership cycle, In the Figure 1.1 “Operator” asks the “Leader” to join or remove a node; “Leader” catches up the “Learner”; “Learner” is promoted to the “Followers” (voters), so the number of voters N changes while the cluster is running.

So, every entity is very much interactive with each other.

2 Overall Description

2.1 Product Perspective

“RaftKV” is a standalone system. It does not depend on or replace another product; it is itself the infrastructure an application would use for replicated storage. A deployment is a set of N node processes, each with its own durable storage (there is no shared disk), and N can change while the system is running. Main goal of this project is to keep the data consistent and available even when some nodes fail, and to let the cluster grow or shrink without stopping it.

Interface	Between	Technology
Client interface	Client application / `raftctl` and any of the N nodes	HTTP/JSON
Node-to-node interface	Cluster nodes (voters and learners)	gRPC / Protobuf
Storage interface	Node and its local disk	WAL file + snapshot files
Deployment interface	Operator and the cluster	Docker Compose, `scripts/`

A write that reaches a follower is never applied locally; the follower redirects the client to the leader.

2.2 User Classes and Characteristics

“RaftKV” has basically 5 types of users.

- Application Client
- Cluster Operator
- Test Harness / Fault Controller
- Developer / Maintainer
- Cluster Node (internal actor)
  - Voter (Leader, Follower, Candidate)
  - Learner

Cluster Node has 2 types - Voter takes part in the elections and is counted for the majority; Learner only receives the log and is not counted. A Learner becomes a Voter when the operator adds a node and the leader promotes it. Application client and operator who fulfill all the requirements can take advantages of the cluster.

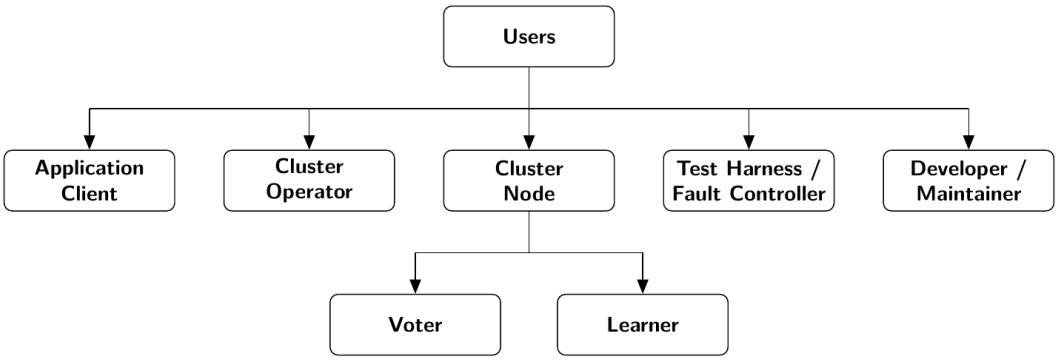


Figure 2.1: type of users

User class	Characteristics	Main activities
Application Client	Programs or users calling the HTTP API	PUT, GET, DELETE keys
Cluster Operator	Runs the cluster and the `raftctl` CLI	Check status, add (and later remove) nodes, so changing N; start/stop containers
Test Harness / Fault Controller	Automated test code	Kill, restart, partition and delay nodes; assert on outcomes
Developer / Maintainer	Team members owning modules	Implement, review and extend modules
Cluster Node (internal actor)	A RaftKV process, one per member of the cluster	Elect leaders, replicate logs, persist state, serve requests, join and be promoted

2.3 Product Functions

"RaftKV" stores all the key-value data of the application in a cluster of N nodes. Before using the main function of the software, the cluster has to be started with its first N nodes and the nodes have to know the cluster address of each other. The functions are:

- Replicated key-value storage with strong consistency for writes, on N nodes (N is configurable).
- Leader election and automatic failover.
- Log replication with majority commit, and in-order application to a key-value state machine.
- Durable state: term, vote and log survive crashes and restarts.
- Snapshots and WAL compaction; InstallSnapshot for nodes that fall far behind.
- **Dynamic membership**: join as learner, catch up, promote to voter (N to N + 1) while the cluster serves requests; node removal (N to N − 1) as a stretch goal.
- A CLI (`raftctl`) for cluster status, join and remove.
- Scriptable fault injection for repeatable failure testing.
- Logging and metrics for observing elections, replication and membership changes.

For N voters the majority is $\lfloor N/2 \rfloor + 1$, and the cluster keeps working while at most $\lfloor (N-1)/2 \rfloor$ voters are down. This is why N is chosen odd in practice: an even N needs a bigger majority without tolerating one more failure.

Voters N	Majority (quorum)	Failures tolerated
3	2	1
4	3	1
5	3	2
general N	$\lfloor N/2 \rfloor + 1$	$\lfloor (N-1)/2 \rfloor$

Table 2.1: Cluster size N, majority and fault tolerance (learners are never counted)

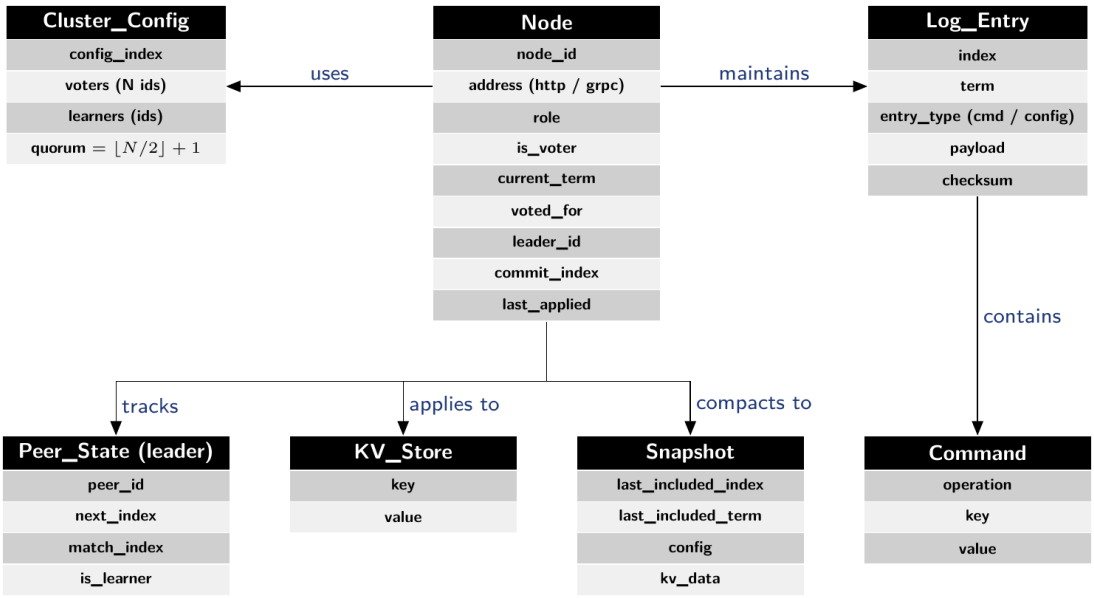


Figure 2.2: Data Flow Diagram

Figure 2.2 shows the data that every node keeps. All nodes have - node_id, address, role, is_voter, current_term, voted_for, leader_id, commit_index and last_applied. The role is one of Follower, Candidate or Leader; is_voter is false while the node is a learner.

Each node maintains a log. Every Log_Entry has an index, a term, an entry_type and a payload. The payload is either a Command (operation PUT or DELETE, key and value) or a cluster configuration change. The configuration (Cluster_Config) lists the voters, so it also gives N, and the learners. Because a configuration change is a log entry, it is replicated and committed like any other entry.

Only the leader keeps a Peer_State for every other member of the configuration: the next_index to send, the match_index known to be replicated, and whether the member is a learner. Committed entries are applied to the KV_Store. A Snapshot records the last_included_index and last_included_term, the configuration in effect at that index and the full key-value data.

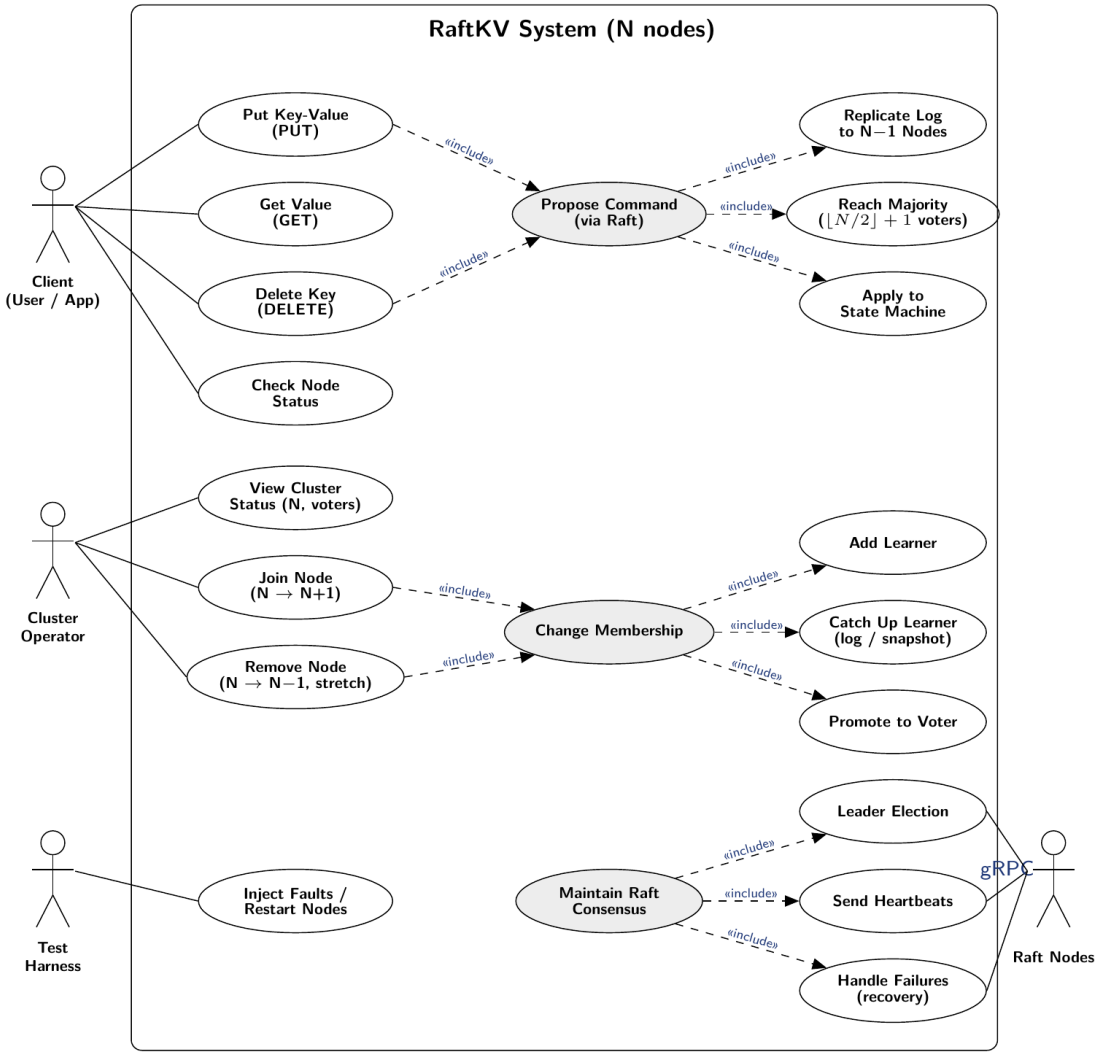


Figure 2.3: Use cases of RaftKV

Figure 2.3 shows the system from the view of each user. A client can put, get and delete a key and check the status of the node it talks to. Put and Delete include “Propose Command”, which means the leader replicates the command to the other N − 1 nodes, waits for a majority of the voters and applies it to the state machine. In the MVP a Get is served from the applied state of the node and does not need a proposal (section 3.2.5). The operator can see the cluster status and can join (or remove) a node; both include “Change Membership”, which adds a learner, catches it up and promotes it to a voter. “Maintain Raft Consensus” is the internal behaviour (leader election, heartbeats, failure handling) in which the other Raft nodes take part over gRPC.

2.4 Operating Environment

The software will be operated in any Operating Environment that can run Docker - Mac, Windows, Linux etc.

- Implementation language: Go. Inter-node communication: gRPC with Protobuf. Client communication: HTTP/JSON.
- Each node runs as an independent Docker container with its own named volume mounted at `/data`. A cluster of N nodes is N containers, so starting one more container (and running `raftctl join`) makes the cluster N + 1; no existing node is restarted.
- Example port layout in the local cluster: node i uses HTTP port 8000+i and gRPC port 9000+i, that is node 1 gRPC 9001 / HTTP 8001, node 2 gRPC 9002 / HTTP 8002, node 3 gRPC 9003 / HTTP 8003, and the same rule for the 4th and 5th node.
- Target network: a low-latency LAN or a Docker bridge network on a single host.

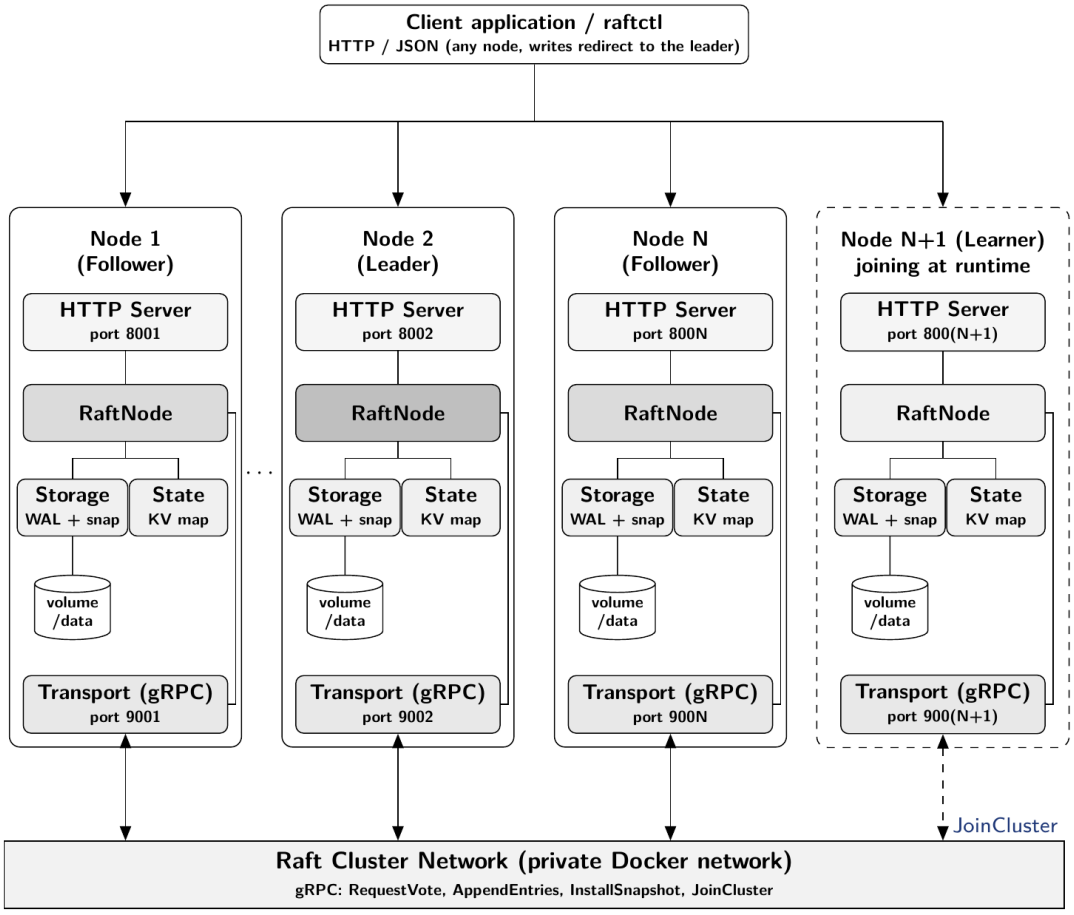


Figure 2.4: Deployment of a cluster of N nodes with a node joining at runtime

Figure 2.4 shows the deployment. There is no fixed number of nodes: Node 1 to Node N are the voters (here one of them is the leader) and Node N + 1 is a new node that has just been started. It talks to the cluster only through the gRPC network, first as a learner, and becomes the (N + 1)-th voter after promotion. Each node keeps its own volume, so a node that is restarted keeps its term, vote, log and snapshot.

2.5 Design

Components. Each node process contains the following modules.

Package	Responsibility
`internal/raft`	Raft core: election, terms, log replication, commit and applied index tracking, persistent Raft state, current configuration
`internal/storage`	WAL and snapshot persistence
`internal/transport`	gRPC server and client
`internal/state`	Key-value state machine (Apply(command) result)
`internal/api`	HTTP handlers for clients
`internal/membership`	Join, promotion and removal logic (changes N)
Fault-injection hooks	Test builds only; wrap the transport
Observability	Logging and metrics

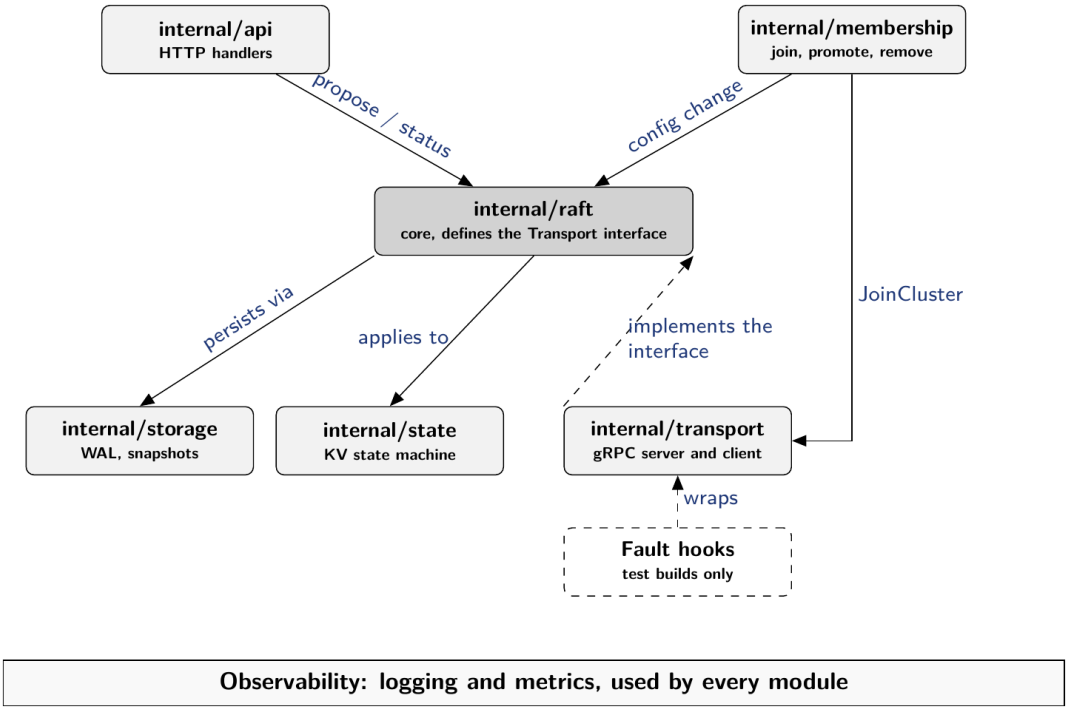


Figure 2.5: Modules of a node and their allowed dependencies

Coupling rules (enforced by code review), see Figure 2.5:

- `internal/raft` may depend on `internal/storage` but not directly on transport, api or membership. Raft talks to transport through an interface that Raft defines (dependency inversion).
- `internal/state` has no knowledge of Raft, transport or networking.
- Only `internal/api` and `internal/membership` may depend on both the Raft core and transport.

Raft state model. The roles are Follower, Candidate and Leader. A node starts as a Follower. A Follower that times out becomes a Candidate; a Candidate that gets a majority of the N voters becomes Leader; any node that discovers a higher term reverts to Follower. The Leader sends periodic heartbeats (empty AppendEntries) to all followers and learners. A Learner is treated as a membership attribute layered on the Follower role, not a fourth Raft state (see section 5.3): a new node enters as a Learner and leaves that attribute when it is promoted to a voter.

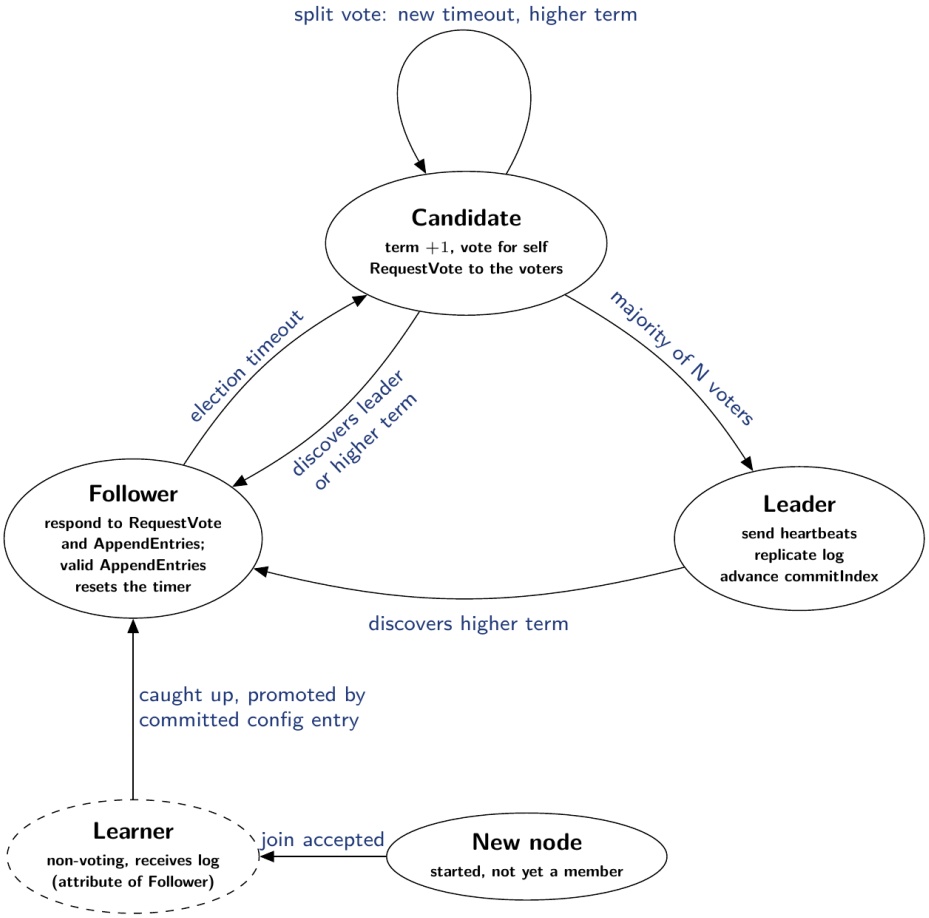


Figure 2.6: Node states and transitions

Timing. The election timeout is randomised in the range 150 to 300 ms and the heartbeat interval is about one tenth of the minimum election timeout (about 15 ms). Both are configurable assumptions (section 5.3).

Write path. Client write activities have 3 steps -

- Request Process
- Commit Process
- Client Response

A client sends a write to any node. A follower redirects the client to the leader. The leader appends the entry to its WAL (fsync), replicates it with AppendEntries to the other N − 1 nodes, advances the commit index once a majority of the voters hold it, applies it to the state machine, and then answers the client. Every client response is one of success, redirect, not-found or error.

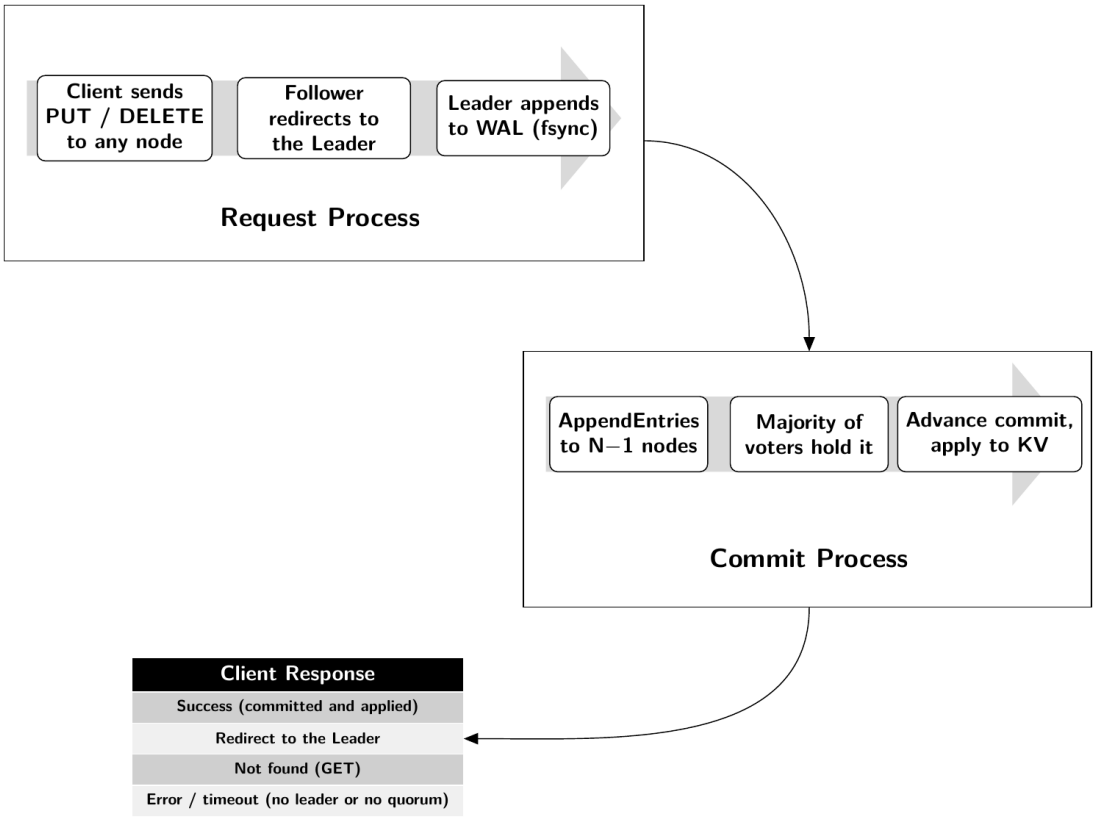


Figure 2.7: Client write activities

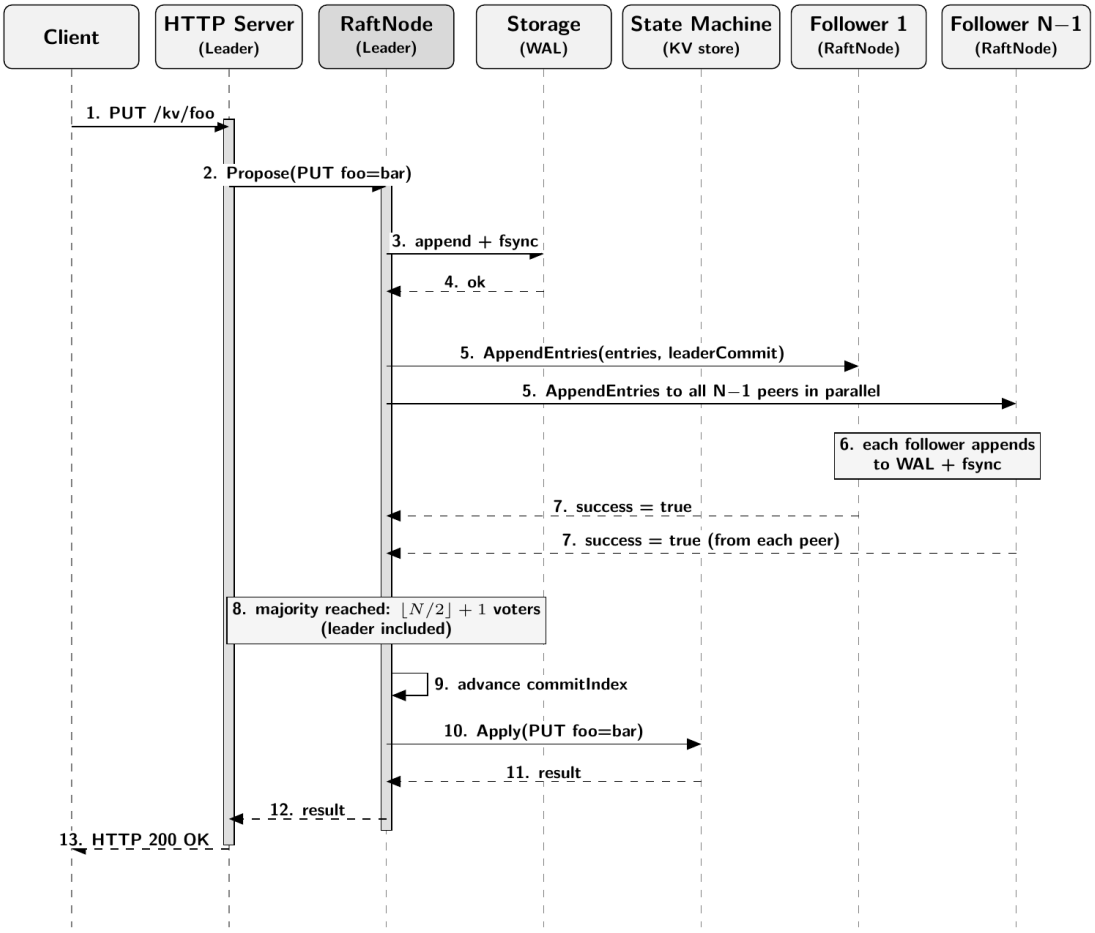


Figure 2.8: PUT request and log replication in a cluster of N voters

Figure 2.8 shows the normal write when the contacted node is the leader. The leader appends the entry to its own log first, then sends it to the other N − 1 nodes in parallel. The entry is committed after a majority, ⌊N/2⌋+1 of the N voters including the leader, have stored it; the answer of the remaining nodes is not needed. If the contacted node is not the leader it redirects the client to the current leader (not shown in the figure).

Dynamic membership. Membership activities have 3 steps -

- Join Process
- Promotion Process
- Cluster Configuration

The operator starts the new node and runs `raftctl join`. The leader adds the node as a learner by a configuration entry in the log, and sends it the snapshot and the log entries it needs while client writes continue. A learner is not counted for the majority, so a slow learner cannot slow down or block commits. When the learner has caught up, the leader promotes it to a voter by a second configuration entry. From that moment N is N + 1 and the majority is recomputed over the new set of voters. Only one membership change is in progress at a time. Removing a node (stretch) is the same path in the other direction.

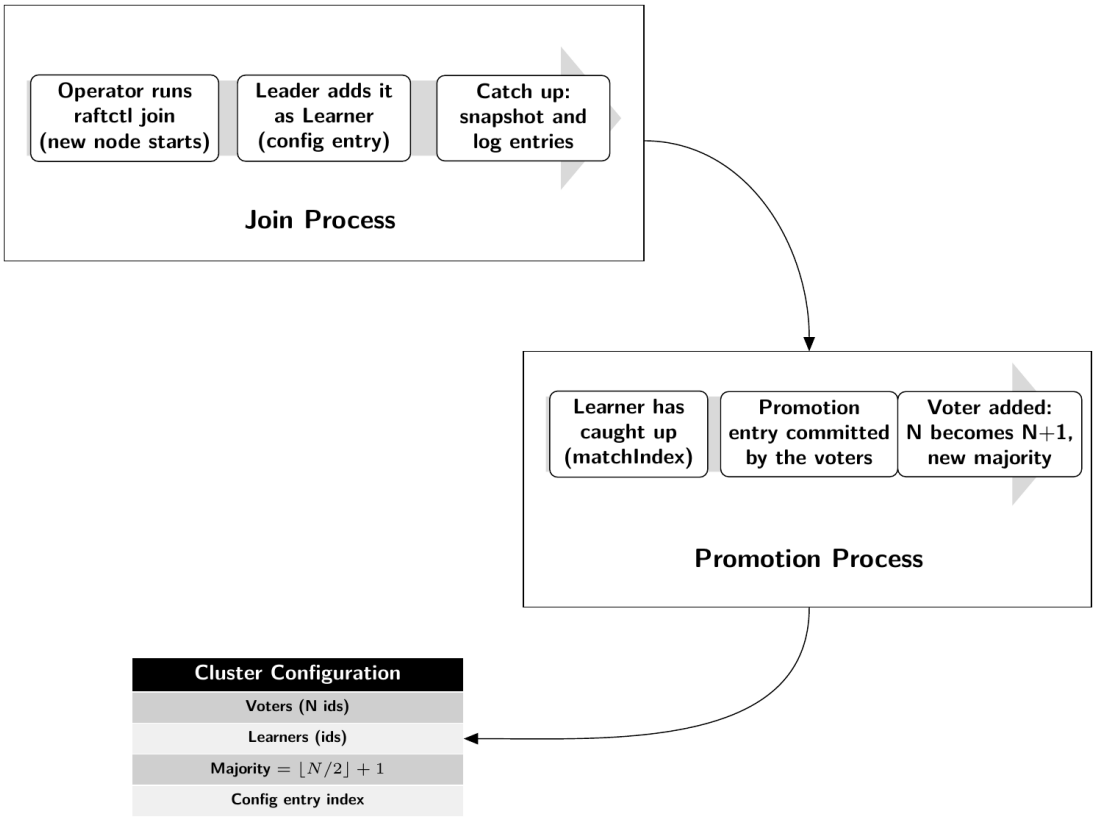


Figure 2.9: Membership activities (a node joins and N grows)

Persistence. One WAL file per node holds Raft metadata (term, vote) and log entries, each record checksummed. Snapshots are periodic full copies of the key-value state together with the configuration in effect, stored as files with a small metadata file pointing at the latest one. After a snapshot the WAL is compacted by rewriting a trimmed file.

Expected behaviour under failure. With N voters the cluster tolerates the failure of up to ⌊(N−1)/2⌋ voters (Table 2.1).

Scenario	Expected behaviour
Leader crash	Followers stop receiving heartbeats, a new election runs, a new leader is elected by a quorum of the remaining voters. Entries that never reached a quorum are lost, which is correct because they were never committed.
Follower crash	No loss of availability while a quorum remains. On restart it catches up through AppendEntries, or through a snapshot if far behind.
Loss of quorum	Fewer than ⌊N/2⌋+1 voters are alive: writes stop committing by design. Writes resume when a quorum is restored.
Network partition	The minority side cannot elect a leader or commit. The majority side continues. After healing, minority nodes see a higher term, step down and re-sync.
Node restart	The node reloads term, vote, log, snapshot and configuration, rejoins as a follower and catches up.
Crash during recovery	Recovery reads checksummed persisted state and does not change it until validated, so it can safely be repeated.
Join while writing	Writes continue. The learner is not counted for the majority, so the commit rule does not change until the promotion entry is in effect.
Learner crash mid catch-up	The learner resumes from its persisted state; the leader continues from the learner's matchIndex.
Leader crash during a membership change	The new leader finishes or drops the change according to the log: the config-change entry is either committed (and recovered like any log entry) or absent. No partial membership state exists.

Activities of the roles. Client activities, operator activities and node activities are shown in Figure 2.10. The operator is the only user that can change N.

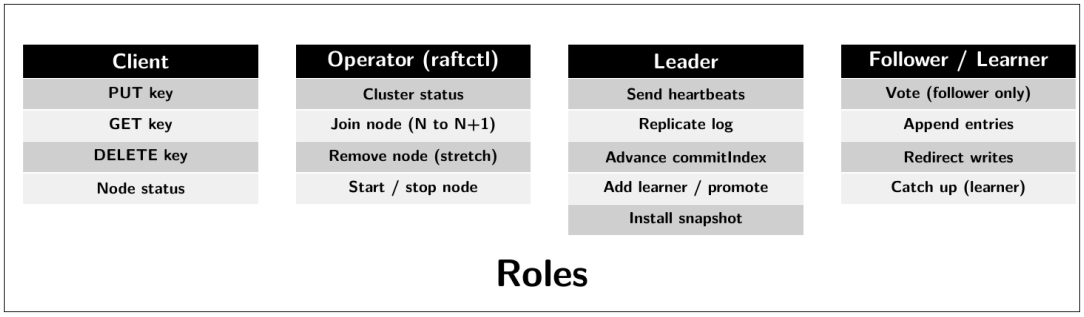


Figure 2.10: Client, operator and node activities

3 System Features

“RaftKV” is a consistency-first replicated store. So the main art of this product is to keep the log of N nodes in agreement, to apply it in order and to let the set of nodes change safely. Correctness of the Raft protocol comes before convenience features.

3.1 Description and Priority

“RaftKV” has features that are main and also some are sub. The MVP features are all necessary for this software; the stretch features begin only after they are done.

The features with priority up to down -

1. Leader Election and Log Replication (MUST) : This is the goal feature of this software. It is the core, and it is done on N nodes with a majority of ⌊N/2⌋+1 voters.
2. Durable Storage and Crash Recovery (MUST) : Term, vote and log survive a crash and a restart.
3. Client Key-Value API with Leader Redirect (MUST) : PUT, GET and DELETE through any node.
4. Snapshotting and Compaction (MUST) : Keeps the log bounded and brings slow nodes up to date.
5. Dynamic Membership (MUST) : Join, learner catch-up and promotion, so N changes at runtime.
6. Fault Injection and Testing Support (MUST) : Repeatable crash, restart and partition tests.
7. Deployment, CLI and Observability (MUST) : Docker Compose cluster of N nodes, `raftctl`, logs and metrics.
8. Node Removal (SHOULD, stretch) : N to N − 1.
9. Linearizable Reads (SHOULD, stretch).
10. Chunked Snapshots, Deeper Metrics, Benchmarking, Optional KV Features (MAY, stretch).

Hard rule: no stretch item may begin until all MVP items are implemented, tested and documented.

3.2 Functional Requirements

The “RaftKV” system is being built with Go, gRPC and HTTP/JSON, and is deployed with Docker.

Back-End - Go (Raft core, storage, state machine, transport, API, membership).
Node-to-Node - gRPC with Protobuf (RequestVote, AppendEntries, InstallSnapshot, JoinCluster).
Client Interface - HTTP/JSON and the `raftctl` command line tool.
Database - no external database; each node uses its own WAL file and snapshot files.
Deployment - Docker and Docker Compose, an N-node cluster.
Quality - Go tests, race detector and a CI pipeline.

Every requirement has an ID. A requirement marked MUST is needed for the MVP, SHOULD is a stretch item that is attempted after the MVP is stable, and MAY is optional. Requirements for the number of nodes N and for dynamic membership are collected in sections 3.2.1 and 3.2.8.

3.2.1 Cluster Size, Quorum and Node Roles

ID	Requirement	Priority
FR-1	The cluster shall consist of N voting nodes, where N is configurable at start-up; 3 to 5 nodes shall be supported.	MUST
FR-2	N shall not be fixed at build or deployment time. The software shall contain no hard-coded cluster size or fixed peer list: the members are taken from the cluster configuration (the bootstrap configuration at the first start, later the configuration recorded in the log or snapshot).	MUST
FR-3	A quorum shall be a majority of the voters of the configuration in effect, that is ⌊ N/2⌋+1 for N voters. The same rule shall be used for elections and for commits, and learners shall never be counted.	MUST
FR-4	A node shall be in exactly one of Follower, Candidate or Leader at any time, and shall start as a Follower.	MUST
FR-5	A learner shall replicate the log like a follower, but shall never vote, never become a candidate, and never be counted for quorum.	MUST
FR-6	Each node shall keep a monotonically increasing current term. A node that sees a higher term in any RPC or response shall adopt it and revert to Follower.	MUST

3.2.2 Leader Election

ID	Requirement	Priority
FR-7	A follower that receives neither a valid AppendEntries from the current leader nor grants a vote within its randomised election timeout shall become a Candidate, increment its term, vote for itself and send RequestVote to all voters.	MUST
FR-8	On RequestVote, a node shall grant at most one vote per term, and only if the candidate’s log is at least as up-to-date as its own (Raft paper, section 5.4.1).	MUST
FR-9	A node’s vote (and term) shall be persisted and fsync’d before the RequestVote response is sent.	MUST
FR-10	A candidate that receives votes from a majority of voters shall become Leader and immediately send heartbeats.	MUST
FR-11	If an election ends without a winner (split vote), candidates shall time out with new random timeouts and start a new election in a higher term.	MUST
FR-12	The Leader shall send periodic heartbeats (empty AppendEntries) to all followers and learners at an interval of about one tenth of the minimum election timeout.	MUST
FR-13	A Leader or Candidate that discovers a higher term shall step down to Follower.	MUST

3.2.3 Log Replication

ID	Requirement	Priority
FR-14	Each log entry shall carry an index, a term and a payload. The payload is either a key-value command or a cluster configuration change.	MUST
FR-15	The AppendEntries RPC shall include prevLogIndex and prevLogTerm. A follower shall reject the request if its log does not contain a matching entry, and shall delete conflicting entries before appending new ones.	MUST
FR-16	The Leader shall track nextIndex and matchIndex for every follower and learner, and shall back off nextIndex on rejection until the logs match.	MUST
FR-17	The Leader shall advance commitIndex only when an entry is stored on a majority of voters and belongs to the Leader's current term (Raft paper, section 5.4.2).	MUST
FR-18	Followers shall advance their commitIndex from the leaderCommit field of AppendEntries.	MUST
FR-19	Every node shall apply committed entries to the state machine strictly in log order, exactly once, with lastApplied never exceeding commitIndex.	MUST
FR-20	Learners shall receive log entries but shall be excluded from the majority calculation.	MUST

3.2.4 Key-Value State Machine

ID	Requirement	Priority
FR-21	The state machine shall implement Apply(command) result for PUT and DELETE commands and shall be deterministic.	MUST
FR-22	The state machine shall have no knowledge of Raft, transport or networking.	MUST
FR-23	The state machine shall be able to produce a snapshot of its full state and restore itself from one.	MUST

3.2.5 Client API

ID	Requirement	Priority
FR-24	Nodes shall expose HTTP/JSON operations to PUT (create or overwrite), GET and DELETE a key. Exact routes and payloads are defined in the API documentation.	MUST
FR-25	A write received by a non-leader shall not be applied locally. The node shall redirect the client to the current leader.	MUST
FR-26	A write shall be acknowledged only after its log entry is committed and applied.	MUST
FR-27	A GET for a key that does not exist shall return a clear not-found result.	MUST
FR-28	When no leader is available or no quorum exists, a write shall fail or time out with an error and shall not report success.	MUST
FR-29	A cluster status operation shall report, at minimum, the node's role, current term and known leader, and the voters and learners of the configuration in effect (so that N can be observed).	MUST
FR-30	MVP reads are served from applied state and are not guaranteed linearizable; this consistency gap is documented.	MUST
FR-31	The system shall provide linearizable reads using read-index or leader-lease.	SHOULD

3.2.6 Persistence and Recovery

ID	Requirement	Priority
FR-32	Each node shall persist currentTerm, votedFor and the log in a per-node WAL.	MUST
FR-33	WAL records shall be checksummed, and shall be fsync'd before any AppendEntries or RequestVote response that depends on them is sent.	MUST
FR-34	On startup a node shall read its WAL and latest snapshot, verify checksums, reconstruct its state and start as a Follower. Records that fail verification shall not be applied.	MUST
FR-35	Recovery shall be re-entrant: a crash during recovery shall not corrupt state, and recovery shall not modify persisted data until it has been validated.	MUST
FR-36	A restarted node shall catch up through AppendEntries, or through InstallSnapshot when the entries it needs have been compacted.	MUST

3.2.7 Snapshotting and Compaction

ID	Requirement	Priority
FR-37	Each node shall periodically snapshot the full key-value state. A snapshot shall record lastIncludedIndex and lastIncludedTerm.	MUST
FR-38	A snapshot shall also record the cluster configuration in effect at lastIncludedIndex, so that the membership (and so N) survives log compaction.	MUST
FR-39	Snapshots shall be written to files, and a small metadata file shall be updated to point at the latest snapshot.	MUST
FR-40	After a snapshot, the WAL shall be compacted up to lastIncludedIndex by rewriting a trimmed WAL file.	MUST
FR-41	The InstallSnapshot RPC shall let the Leader bring a follower or learner up to date when the entries it needs are no longer in the Leader's log. The follower shall discard the log the snapshot covers.	MUST
FR-42	Snapshot transfer shall be possible in chunks (streaming). The MVP may send a snapshot in one piece.	MAY

3.2.8 Dynamic Membership

Dynamic membership means that the set of voters, and so the value of N, can change while the cluster is running and serving clients. A membership change is a log entry, so it is replicated, committed and recovered like any other entry.

ID	Requirement	Priority
FR-43	A new node shall be able to request to join the running cluster, without restarting any existing node (JoinCluster RPC, triggered by the `raftctl join` command).	MUST
FR-44	A join request that reaches a non-leader shall be redirected to the Leader.	MUST
FR-45	The Leader shall add the joining node as a learner. The configuration change shall be a log entry with the same commit-or-not semantics as any other entry.	MUST
FR-46	The Leader shall transfer the needed snapshot and log entries to the learner, and the learner shall catch up while client writes continue.	MUST
FR-47	Once a learner has caught up, it shall be promoted to a voter through a committed configuration-change entry; N then increases by one.	MUST
FR-48	At most one membership change shall be in progress at a time. A further change request shall be rejected or deferred until the previous configuration entry is committed.	MUST
FR-49	During and after a change, every election and commit decision shall use one well-defined voter set, so that two disjoint majorities can never exist (no split-brain).	MUST
FR-50	A membership change shall not interrupt client reads and writes: the cluster stays available while a node joins.	MUST
FR-51	Changes shall be repeatable: N may grow step by step (for example 3 to 4 to 5) by repeated join and promotion, and a promoted node shall take part in elections and in the majority from the moment its promotion takes effect.	MUST
FR-52	Cluster membership shall survive a restart of the entire cluster.	MUST
FR-53	If a learner crashes during catch-up, it shall resume from its persisted state after restart, and the Leader shall continue from the learner’s matchIndex using normal replication retry.	MUST
FR-54	A crash during a membership change shall leave no partial membership state: the change is either committed or absent.	MUST
FR-55	The join endpoint shall be protected by a shared-secret or static bearer token.	SHOULD
FR-56	An operator shall be able to remove a voter safely (N to N − 1), including the case where the Leader removes itself.	SHOULD
FR-57	A node that has been removed shall stop voting and shall not start elections.	SHOULD

3.2.9 Fault Injection and Testing Support

ID	Requirement	Priority
FR-58	Raft shall reach the network only through a transport interface, so that a test-only transport decorator can drop, delay and reorder messages and partition groups of nodes. Fault injection shall not appear as conditional code in Raft core.	MUST
FR-59	The harness shall support process-level crash and restart in two tiers: in-process (stop a Raft instance, discard memory, reload from storage) and Docker-based (docker stop / docker start).	MUST
FR-60	Fault scenarios shall be data-driven or scriptable (for example, ‘at tick 5 kill the leader; at tick 8 restart it’), so one harness can drive many scenarios.	MUST
FR-61	The harness shall cover at least leader crash, follower crash, network partition (majority and minority behaviour) and node restart, and shall assert convergence, no data loss, no split-brain and correct quorum behaviour.	MUST
FR-62	Join shall be tested for at least: normal join, join during writes, and restart during catch-up.	MUST
FR-63	The tests shall run clusters of more than one size (at least N = 3 and N = 5) and shall include a test in which the cluster grows from N to N + 1 under client load, asserting no data loss and a correct majority afterwards.	MUST

3.2.10 CLI, Deployment and Observability

ID	Requirement	Priority
FR-64	A separate `raftctl` binary shall wrap the HTTP API for cluster status and join, and for remove once FR-56 exists.	MUST
FR-65	A `docker-compose.yml` shall bring up a configurable N-node cluster. Each node shall run in its own container with its own named volume for durable storage.	MUST
FR-66	A script in `scripts/` shall add an additional node (for example the 4th or 5th) to an already-running compose cluster, demonstrating the join protocol on the deployed system.	SHOULD
FR-67	Nodes shall log role changes, elections, replication progress, snapshots and membership events, so the join process can be followed end to end.	MUST
FR-68	Nodes shall expose basic metrics. Deeper metrics (commit latency histograms, catch-up lag) are optional.	MUST (basic), MAY (deep)
FR-69	The system may offer watches, TTL/leases, compare-and-set and simple transactions. These are attempted only after all other items are done.	MAY

4 Other Nonfunctional Requirements

4.1 Performance Requirements

“RaftKV” is used as a consistency-first store, so durability and correctness are more important than speed.

ID	Requirement	Priority
NFR-1	Consistency over availability: with fewer live voters than a quorum (⌊ N/2⌋+1), writes shall stop committing. This is intended behaviour, not a defect.	MUST
NFR-2	With N voters the cluster shall keep accepting and committing writes while at most ⌊ (N−1)/2⌋ voters are unavailable (Table 2.1). This shall hold for every N the cluster has, including after nodes have joined at runtime.	MUST
NFR-3	Timing shall follow the Raft guidance: heartbeat interval much smaller than the election timeout, with randomised timeouts to avoid repeated split votes.	MUST
NFR-4	Durability takes priority over latency: the fsync before a dependent response (FR-33) shall not be skipped for speed.	MUST
NFR-5	No numeric throughput or latency target is set. A simple benchmark suite with a short write-up is optional.	MAY

4.2 Security Requirements

No one without the cluster address and, for joining, the join token can add a node to the cluster. One particular user of a section only can perform his/her particular actions: clients use the key-value API, the operator uses `raftctl`, and nodes use gRPC.

ID	Requirement	Priority
NFR-6	The join endpoint shall require a shared secret or static bearer token (see FR-55).	SHOULD
NFR-7	TLS/mTLS between nodes, RBAC, encryption at rest and compliance audit logging are not provided. The cluster is assumed to run on a trusted local network.	OUT OF SCOPE

Because unrestricted joining lets an attacker observe replicated data (and vote, if later promoted), the join token is the most valuable low-cost protection.

4.3 Software Quality Attributes

In the development phase also testing and review are continued, so that the quality of the software is maintained and all the requirements are fulfilled. Unit, integration and failure tests are required.

ID	Attribute	Requirement
NFR-8	Safety	The Raft safety properties shall never be violated: Election Safety (at most one leader per term), Leader Append-Only, Log Matching, Leader Completeness, and State Machine Safety. Each shall be covered by a test.
NFR-9	Reliability	Committed data shall survive the crash of any minority of nodes and a restart of the full cluster.
NFR-10	Configurability	The cluster size N and the membership shall be set by configuration and by the join protocol, never by a code change. The correctness tests shall pass for every supported N (at least 3 and 5), and quorum and fault tolerance shall follow Table 2.1.
NFR-11	Concurrency	`go test -race ./...` shall pass clean.
NFR-12	Modularity	The coupling rules in section 2.5 shall hold and be checked in code review.
NFR-13	Testability	Every Raft or membership feature shall have unit tests, integration tests in a running multi-node cluster, and failure tests; happy-path tests alone are not enough.
NFR-14	Automation	CI shall run build, vet, test and race checks on every pull request.
NFR-15	Documentation	The SDD, API documentation, the required Mermaid diagrams and ADRs for the design decisions shall be maintained.
NFR-16	Observability	Relevant logs and metrics shall exist for each feature.
NFR-17	Simplicity	The design shall follow the Raft paper closely, use a single WAL file per node, and avoid unnecessary distributed-systems features.

Definition of Done (every feature). Merged through a reviewed pull request (two reviewers for `internal/raft` and `internal/membership`); unit, integration and failure tests exist; `go test ./...` and `go test -race ./...` pass in CI; documentation (API doc, architecture doc or ADR) is updated; logs and metrics are present.

4.4 Business Rules

- “RaftKV” is a university project. Its purpose is a correctly engineered, dynamically reconfigurable, strongly consistent N-node Raft key-value store.
- No feature outside this SRS shall be added. Scope shall be re-checked against section 1.3 and section 5.2 before adding anything (the “make it more like etcd” temptation).
- Stretch goals begin only after the MVP is implemented, tested and documented. Their order is: node removal, linearizable reads, chunked snapshots, deeper metrics, benchmarking, optional KV features.
- Membership work starts only after election and replication tests pass.
- Assumptions shall be marked as such and recorded in an ADR.

5 Other Requirements

5.1 Maintenance and Evolution

"RaftKV" needs maintenance as it is a long process software. The module boundaries in section 2.5 are intended to let each module be changed or replaced independently (for example, swapping the transport or storage layer). It will need refactoring and further the requirements can be changed. This SRS is a living document: changes to a requirement shall be reflected here, in the SDD and in the ADRs.

5.2 Out of Scope

TLS/mTLS between nodes, RBAC, encryption at rest, audit logging as a compliance feature, Kubernetes manifests, cloud deployment automation, multi-host orchestration, segmented or rotating WAL files, and any distributed-systems feature not named in this document.

5.3 Assumptions and Items to Confirm

- **Learner is not a fourth Raft state.** It is modelled as a membership attribute on a Follower. The design document flags this as defensible but non-obvious; confirm with the team and instructor.
- **Range of N.** The MVP supports 3 to 5 voters (FR-1). The design places no fixed upper bound on N, and the exact largest N that is tested is to be confirmed.
- **How membership changes are made.** Requirements FR-48, FR-49 and FR-38 follow the usual Raft approach of changing one server at a time through log entries, and recording the configuration in snapshots. They, together with FR-2, FR-3, FR-44, FR-50, FR-51, FR-57, FR-63, the voter and learner lists in FR-29, NFR-2 and NFR-10, were added or extended in this version to make N and dynamic membership explicit. Confirm them against the design document.
- **Timing values.** The 150 to 300 ms election timeout and a heartbeat of about one tenth of the minimum are assumed. They are tuning decisions made at implementation time.
- **Implementation status.** Phases 1 and 2 are implemented, but the phase plan and the code were not available when this version was written. Add a Phase/Status column to the tables in section 3.2 once the mapping from phases to requirements is confirmed.
- **Read semantics.** What FR-30 means in detail (which node serves a GET, and how stale it may be) depends on the consistency discussion in the design document, which was not available.
- **API details.** Exact HTTP routes, status codes and the redirect mechanism (FR-24, FR-25) are left to the API documentation. The route used in Figure 2.8 and the ports in section 2.4 are examples.
- **Storage details.** The WAL record format, snapshot file naming and how the snapshot pointer is updated safely (FR-33, FR-39) are not fixed here.
- **Promotion and removal protocols.** The criteria for judging that a learner has caught up (FR-47) and the removal protocol (FR-56) come from design sections that were not available.
- **Safety invariants.** The design document's invariant list was cut off in the source. NFR-8 uses the five properties from the Raft paper.
- **Single WAL file.** One WAL file per node, rewritten on compaction, is a deliberate simplification for this project's data volumes.

- **Source coverage.** This SRS was derived from the design-document sections on architecture, state machine, failure handling, storage, security, deployment, fault injection, MVP, stretch goals, risks and Definition of Done. Requirements and sections not seen (the original requirements list, other module specifications, the join, promotion and removal protocols, API and testing sections) should be reviewed against this document.
- **Authors.** Team member names, IDs and the instructor's name on the cover page are placeholders.

5.4 Glossary

Term	Meaning
Raft	A consensus algorithm that keeps a replicated log consistent across nodes
N	The number of voting nodes in the cluster. It is configurable, and it changes by one each time a node is promoted to voter or removed
Term	A logical clock period in Raft; each term has at most one leader
Leader / Follower / Candidate	The three Raft roles
Learner	A non-voting member that replicates the log and is not counted for quorum
Voter	A member that votes in elections and counts toward quorum
Quorum	A majority of voters, ⌊ N/2⌋+1 for N voters
Configuration	The set of voters and learners in effect, recorded as configuration entries in the log and in snapshots
Dynamic membership	Changing the configuration, and so N, while the cluster is running (join, promote, remove)
WAL	Write-ahead log: durable, checksummed record of Raft metadata and log entries
Snapshot	A point-in-time copy of the full key-value state
commitIndex / lastApplied	Highest log index known to be committed / highest index applied to the state machine
nextIndex / matchIndex	The Leader's per-follower record of the next entry to send / the highest entry known to be replicated
Split-brain	Two nodes both acting as leader for the same data; must never occur
Linearizable	Behaves as if all operations happened one at a time in a real-time order